FRONTEND ANGULAR · TAKE-HOME TASK

Kitchen Orders Board

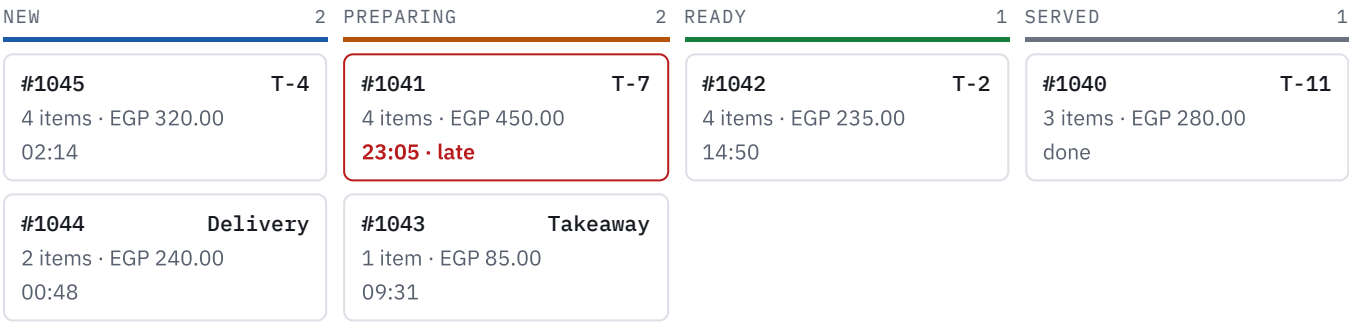
Thanks for your interest in the role. Next step: build a small Angular app that restaurant staff use to track orders from the moment they are placed until they reach the table.

EXPECTED EFFORT	DEADLINE	STACK	BACKEND
5–7 hours	**4 days from receipt**	**Angular 17+**	**json-server (provided)**

Attached: `db.json` (mock data) and `README.md` (quick start).

What you'll build

A board with one column per order status. Staff move orders forward, open an order to see what's in it, and create new orders. The board refreshes on its own so the kitchen always sees the latest state.



A rough idea of the layout, not a design to copy. The visual design is up to you. Card totals here are subtotals.

Requirements

1. **Orders board** `/orders`
   - Four columns: New → Preparing → Ready → Served, each showing its count.
   - Each card shows order number, table number or order type, item count, total in EGP, and time elapsed since it was created (updates every second).
   - Orders that are not Ready yet and are older than **20 minutes** are marked as late.
2. **Search and filters**
   - Search by order number or table number, and filter by type (dine-in, takeaway, delivery).
   - Store filter state in URL query params, so a refresh or a shared link keeps the same view.
   - Debounce the search input.
3. **Move an order forward**
   - One action on each card moves it to the next status (`PATCH /orders/:id`).
   - Update the UI right away. If the request fails, undo the change and show a clear error message.
4. **Order details** `/orders/:id` `lazy-loaded`
   - Item names, quantities, notes and line totals, joined from the `menu` collection.
   - Price breakdown: subtotal, 12% service (dine-in only), 14% VAT on subtotal + service, grand total.
   - Handle an order id that doesn't exist.

5 **New order form** `/orders/new` `lazy-loaded`

- Reactive form with a dynamic list of items (add/remove rows): menu item, quantity, optional note.
- Validation: at least one item; quantity 1–20; table number (1–40) required for dine-in; Egyptian mobile number required for delivery (`01` + 0/1/2/5 + 8 digits). Write at least one of these as a custom validator.
- New orders start as `new`, get the current time as `createdAt`, and take the next order number.
- Show the running total while the user fills the form. Block double submission.

6 **Auto-refresh**

- Reload the board every 15 seconds without flicker and without starting a new request while one is still in progress.
- Stop polling when the user leaves the page. No leaked subscriptions.

7 **States and layout**

- Clear loading, empty and error states on every screen.
- Usable at 375px wide. On small screens, columns can stack or scroll horizontally, whichever you think works better.

Technical constraints

- Angular 17 or newer, standalone components, TypeScript `strict` mode on.
- Use Signals, RxJS, or both. Be ready to explain why you chose what you did.
- A UI library (PrimeNG, Angular Material) is fine for individual controls. Please don't start from a ready-made admin template.
- Keep API access in services. Components should not call `HttpClient` directly.

Bonus `optional`

Pick one or two if you have time. Doing the core requirements well matters more than doing many bonuses.

- Meaningful unit tests: a validator, the price calculation, one component.
- Arabic / English toggle with correct RTL layout.
- `OnPush` change detection throughout.
- Pause polling while the browser tab is hidden.
- Deployed demo (Vercel, Netlify, GitHub Pages) with a mocked API.

Mock API

Put the attached `db.json` in your project root and run:

```
npx json-server@1.0.0-beta.3 db.json --port 3000
```

This gives you `GET/POST /orders`, `GET/PATCH /orders/:id`, `GET /menu`, and filters such as `/orders?status=new`. Order totals are not stored. Calculate them from `menu` prices. The seed orders have timestamps in the past, so they will all show as late. That is expected.

Order fields

FIELD	TYPE / VALUES
`id`	string
`number`	number, e.g. 1041
`type`	`dine-in` · `takeaway` · `delivery`
`table`	number 1–40, or null
`phone`	string, or null
`status`	`new` · `preparing` · `ready` · `served`
`createdAt`	ISO 8601 string
`items`	array of `{ menuId, qty, note }`

Example order

```
{
  "id": "1041",
  "number": 1041,
  "type": "dine-in",
  "table": 7,
  "phone": null,
  "status": "preparing",
  "createdAt": "2026-09-27T09:22:00.000Z",
  "items": [
    { "menuId": "m2", "qty": 2,
      "note": "no garlic sauce" },
    { "menuId": "m6", "qty": 2, "note": "" }
  ]
}
```

Check your price calculation

ORDER	TYPE	SUBTOTAL	SERVICE 12%	VAT 14%	TOTAL (EGP)
#1041	dine-in	450.00	54.00	70.56	574.56
#1044	delivery	240.00	0.00	33.60	273.60

What to send back

- [ ] A link to a **public GitHub repository**. Please commit as you work. We want to see the real history, not one "final" commit.
- [ ] A `README.md` with setup steps, the main decisions you made and why, what you would improve with more time, and anything you left unfinished.
- [ ] Optional: a deployed link, or a screen recording of 3 minutes or less.

Reply to the message this task came with, and include the links.

How we review it

AREA	WHAT WE LOOK AT	WEIGHT
Angular structure	Feature folders, routing, lazy loading, standalone components, clear separation between services and components	25%
State and async	Correct RxJS/Signals use, polling without overlap, optimistic update and rollback, no leaks	20%
Forms	FormArray handling, custom validators, helpful error messages, correct totals	15%
Code quality	Strict typing, readable names, small focused components, no dead code	15%
UX	Loading, empty and error states; responsive layout; keyboard and focus basics	15%
README and git	Clear explanation of decisions, meaningful commits	10%

Ground rules

- **AI tools are allowed.** If you use them, you are responsible for every line. In the follow-up call we'll ask you to walk through your code and make a small live change. Code you can't explain counts against you.
- Please don't spend much more than 7 hours. If you run out of time, list what's missing in the README. A smaller app that's finished and clean beats a bigger one that's half done.
- If a requirement is unclear, make a reasonable assumption and write it down in the README.

After you submit

We'll review your code within a few working days. If it's a good match, we'll invite you to a 45-minute call to go through your code, talk about your decisions, and make one small change together.

Good luck. We look forward to seeing what you build.